ORACLE

Filesystems & Memory Reclaim

Kernel Recipes

Matthew Wilcox
Technical Advisor
Oracle Linux Development
2025-09-24



What is memory reclaim?

- Most code which allocates memory uses it for a short time and then frees it again
- Some code likes to cache memory it has allocated in case it's useful
  - Page cache, dentry cache, inode cache, slab allocator
- Caches need to register shrinkers so they can free memory
- Linux can reclaim both directly and in the background



Copyright © 2025, Oracle and/or its affiliates

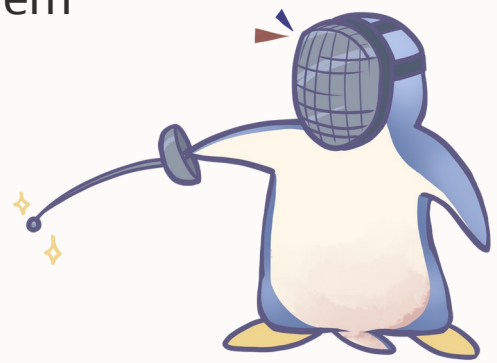
Why do filesystems have problems with memory reclaim?

- Freeing filesystem data structures may be complicated
- Filesystems may need to take locks they already hold

```
mutex_lock(my_lock);
kmalloc(256, GFP_KERNEL);
→ enter reclaim
mutex_lock(my_lock);
```

- Fortunately we have lockdep to catch these problems before users hit them
- This one is easy to solve:

```
mutex_lock(my_lock);
flags = memalloc_nofs_save();
kmalloc(256, GFP_KERNEL);
→ enter reclaim, will not reenter filesystem
memalloc_nofs_restore(flags);
mutex_unlock(my_lock);
```



Copyright © 2025, Oracle and/or its affiliates

A new bug report

- Actually three in quick succession:
  - https://lore.kernel.org/all/CALm_T+3j+dyK02UgPiv9z0f1oj-HM63oxhsB0JF9gVAjeVfm1Q@mail.gmail.com/
  - https://lore.kernel.org/all/CALm_T+2cEDUJvjh6Lv+6Mg9QJxGBVAHu-CY+okQgh-emWa7-1A@mail.gmail.com/
  - https://lore.kernel.org/all/20250326105914.3803197-1-matt@readmodwrite.com/
- Both `fat` and `ext4` filesystems involved
- Both direct and background reclaim involved
-



Copyright © 2025, Oracle and/or its affiliates

A new bug report

```
WARNING at __alloc_pages_slowpath mm/page_alloc.c:4240
Call Trace:
 alloc_pages_mpol_noprof mm/mempolicy.c:2269
 __filemap_get_folio mm/filemap.c:1951
 sb_bread include/linux/buffer_head.h:346
 fat12_ent_bread fs/fat/fatent.c:86
 fat_truncate_blocks fs/fat/file.c:394
 fat_free_eofblocks fs/fat/inode.c:633
 fat_evict_inode fs/fat/inode.c:658
 evict fs/inode.c:796
 prune_icache_sb fs/inode.c:1033
 super_cache_scan fs/super.c:223
 shrink_slab mm/shrinker.c:664
 do_try_to_free_pages mm/vmscan.c:6277
 try_to_free_pages mm/vmscan.c:6527
 __perform_reclaim mm/page_alloc.c:3929
 __alloc_pages_direct_reclaim mm/page_alloc.c:3951
 __alloc_pages_slowpath mm/page_alloc.c:4382
 pte_alloc_one arch/x86/mm/pgtable.c:33
```



Copyright © 2025, Oracle and/or its affiliates

Stack trace analysis

- We set `PF_MEMALLOC` in `__perform_reclaim()`
- We use `GFP_NOFAIL` in `sb_bread()`
- These are individually reasonable, but the combination is Not Allowed
  - It's a bad idea to allocate memory in order to free memory
-



Copyright © 2025, Oracle and/or its affiliates

Let's look for solutions

- Allow `PF_MEMALLOC` to access the atomic reserves
  - With large block size devices, we might need a high-order folio to be allocated
- Use mempools to reserve enough memory
  - The amount of memory needed cannot be known ahead of time
- Pin the needed memory in advance
  - As above … and filesystems might need to run a transaction to evict an inode
- Only allow inode reclaim from kswapd context
  - One of these bug reports is from kswapd
- Allow `→evict_inode()` to fail
  - Other parts of the kernel will wait for an inode in this state to be evicted
- Avoid evicting dirty inodes in `PF_MEMALLOC`
  - I haven't asked anyone about this idea yet



Copyright © 2025, Oracle and/or its affiliates

The preferred solution

- XFS does not suffer from this problem
  - It has different inode lifetime rules from other Linux filesystems
- Converting the Linux VFS to use the same rules as XFS solves two problems
  - I don't know enough to implement this solution
- But I can defer actually freeing the inode to a workqueue
  - This will run in a thread that doesn't have PF_MEMALLOC set
- Isn't this cheating?
  - Yes
- Does conference-driven development work?
  - Maybe



Copyright © 2025, Oracle and/or its affiliates

Other contretemps between fs and mm



- Should reclaim writeback dirty file pages?
  - Removed from XFS in 2021 (v5.15), removed everywhere in May 2025 (v6.16)
- Should `kmalloc()` return 512-byte aligned memory?
  - It does, and now slab is willing to guarantee it
- How hard should `kvmalloc()` try before resorting to `vmalloc()`?
- What should the limited number of folio flags be used for?
  - locked/uptodate/writeback/accessed/lru/active/referenced/dirty/private/private_2/owner/owner_2/...
- What should the refcount of a fs folio be?
  - MM needs to know this to determine if it can split/migrate/... a folio
- The stalled migration from `GFP_NOFS` to `memalloc_nofs_save()`
-

Copyright © 2025, Oracle and/or its affiliates



ORACLE